INTRODUCTION TO OPTIMIZATION TECHNIQUES

IA-2 RESEARCH PAPER DRAFT

Research Gap, Proposed Methodology, Experimental Evaluation & Results

Group ID	OPT-09
Student 1	Razzan Tambe | 15
Student 2	Saurabh Tripathi | 04
Student 3	Vivek Sharma | 12
Research Title	Recipe-Guided Iterated Local Search around Sequential Linear Programming for the Fixed-Demand Pooling Problem: an Extension of our IA-1 Filter-SLP Implementation

Note: IA-2 extends our IA-1 work [15]: the IA-1 files qq_model.py and slp.py (re-implementation of Grothey & McKinnon, 2023) are used **unchanged** as the baseline, and the proposed method is a new module (rg_ils.py) built on top of them.

Abstract

The pooling problem blends raw materials through a few shared bins into many products with quality limits. The bin compositions are decision variables, so the quality constraints are bilinear and the problem is nonconvex with many local optima. In IA-1 we re-implemented the filter Sequential Linear Programming (SLP) method of Grothey and McKinnon [1] on their qq-formulation in Python with the open-source HiGHS LP solver, and reproduced the best known af-1 objective (296.44604). Running the same IA-1 code on all eleven af-series instances showed its weakness: with random starting points, 0% of the runs were Good (within 0.2% of the best known value) on af-2, af-3 and af-6-fs, 5% on af-6-f and af-7, and only 60–75% of the runs were feasible on af-4, af-6-f and af-7.

In IA-2 we keep the IA-1 model (qq_model.py) and SLP solver (slp.py) unchanged and add one module that changes where every SLP run starts and what happens to its end point. The method, **RG-ILS-SLP** (recipe-guided iterated local search around SLP), uses three LP sub-problems that follow from the bilinear structure: a per-product *diet relaxation* (ideal recipe and a lower bound), a *product LP* with the bins fixed and a *bin LP* with the bin shares fixed. They give cost-aware feasible starts, repair and polish every SLP end point, and drive an iterated local search that re-seeds bins with the ideal recipes of the products that lose most from bin sharing (regret). Candidate moves are ranked with short 10-iteration SLP probes.

Over 3 seeds × 20 runs on all eleven instances, the proposed method raises the mean %Feas of our IA-1 baseline from 87.9% to 96.1% and the mean %Good from 43.6% to 62.0%. On the six hard instances (where the published SLP is Good in at most ~20% of the runs) %Good rises from 6.4% to 33.9%, against 14.8% for the published CPLEX-based SLP, and the expected time to a Good solution is 5.0× shorter on the hard instances where both methods find one. The diet relaxation gives a lower bound for af-1 (295.932) equal to Baron's bound, in milliseconds. Limitations: af-6-f (the paper's af-6) is not solved to 0.2% by either method, easy instances become slightly slower, and af-7 stays just below the published %Good.

Keywords: Nonconvex optimization; pooling problem; animal-feed blending; sequential linear programming; LP relaxation; iterated local search; time-to-quality

1. Introduction

Blending problems are among the oldest applications of linear programming. If products are mixed directly from raw materials, the quality constraints are linear. In feed mills and refineries, however, raw materials often pass first through a few *mixing bins* (pools) that several products share. The bin compositions then become

variables, and the quality of a product is a bin composition multiplied by that bin's share in the product. This bilinear term turns an LP into a nonconvex nonlinear program: the pooling problem [7].

Grothey and McKinnon [1] showed that a trust-region filter SLP method on their qq-formulation, restarted from many random points, finds good solutions faster than SQP, interior-point methods and the global solver Baron. In IA-1 our group re-implemented this method (Python, NumPy/SciPy, HiGHS instead of CPLEX), validated it on af-1, and recorded where it struggles. IA-2 starts from exactly that code and that evidence.

1.1 Problem Context and Motivation

A feed mill must produce fixed tonnages t_p of products p. Every product has lower and upper limits on N nutrients. Raw materials either go through M shared bins or are added directly ('straights'). The goal is minimum total raw-material cost. A good set of bin compositions has to serve many products at once, so the problem has many local optima.

SLP solves a sequence of LPs linearised at the current point. Each run is therefore fast, but it only reaches the local optimum nearest its start. With uniform random starts, few runs land in a good basin on the harder instances, and some runs stop at a local minimiser of the constraint violation (infeasible). A better method should (a) end feasible almost always, (b) start in good basins more often and (c) reuse what earlier runs found, while the SLP itself stays as it is.

1.2 Objectives

O1. Use the IA-1 filter-SLP code unchanged as the baseline and measure its %Feas, %Good, Ti, It and ETiGood on all eleven af-series instances of [1], with 3 seeds × 20 runs.

O2. Derive LP sub-problems from the bilinear structure of the qq-formulation that give feasible, cost-aware starts and repair or polish SLP end points.

O3. Replace independent random restarts with a recipe-guided iterated local search that builds on the incumbent.

O4. Compare the proposed method with the baseline, the published SLP-CPLEX results and the IA-1 notebook. Use the paper's metrics plus significance tests and an ablation study.

O5. Deliver reproducible code that extends the IA-1 files without changing them.

1.3 Contributions of the Work

- **Diet relaxation (new auxiliary formulation):** one LP per product that ignores bin sharing. It gives each product's ideal recipe and a valid lower bound on the qq-optimum, equal to Baron's bound on af-1.
- **Exact LP repair and polish (new):** with the bins fixed, or with the bin shares fixed, the qq-constraints become linear. Alternating the two LPs makes approximate points exactly feasible and never increases the cost.
- **Recipe-clustering initialisation (modified initialisation):** weighted k-means++ on the ideal recipes gives the initial bin compositions, instead of uniform random proportions.
- **Regret-guided ILS with SLP probes (modified search strategy / hybridisation):** a bin is re-seeded with the ideal recipe of a product that pays a high price for bin sharing. Four such moves are probed with a 10-iteration SLP, and the best one is continued.
- **Evaluation beyond IA-1:** all 11 instances × 3 seeds × 20 runs, Fisher and Mann–Whitney tests, and a two-part ablation. IA-1 reported 20 runs on af-1 only.

2. Related Work

IA-1 reviewed six papers [1]–[6] on nonlinear and nonconvex optimisation. We summarise those that shape the gap and add the pooling literature on alternating LPs and neighbourhood search [7]–[9], the iterated-local-search framework [13] and k-means++ seeding [14], on which the extension builds.

2.1 Selected Baseline Paper

Grothey and McKinnon [1] study the pooling problem with fixed demands from the animal-feed industry. Their **qq-formulation** uses only proportions: λ_{mi} (share of raw material i in bin m), μ_{pi} (straight share of i in product p) and μ_{pm} (share of bin m in product p), together with the bin and product qualities v_{mn}, d_{pn} and unit costs c^M_m, c^D_p. The only nonlinear terms are $\mu_{pm}v_{mn}$ and $\mu_{pm}c^M_m$.

A trust-region SLP with a Fletcher–Leyffer filter [11] is solved from many random points (CPLEX primal simplex with hot starts). Over 500 runs per instance, SLP-CPLEX takes 0.5–14.7 s per run and is Good in 4–100% of the runs. Its expected time to a Good solution (ETiGood) is on average 863×, 507× and 20× smaller than for FilterSQP, IPOpt and Baron.

Our IA-1 implementation of this method (qq_model.py, slp.py) treats price as an extra attribute so that (5c) and (5g) form one family of bilinear rows. It scales every attribute by the median of its positive entries, tightens the bounds of v and d, and uses an ∞-norm trust region relative to each variable's range ($\rho_0 = 0.5$, $\rho_{max} = 1$). Filter parameters are $\beta = 0.99$, $\gamma = 0.01$, $\sigma = 0.1$, with at most 400 iterations. Restoration is block-wise (exact LPs with μ or (λ, v) fixed) followed by an elastic ℓ_1 LP, and the LPs are warm-started in HiGHS.

2.2 Comparison with Existing Approaches

Table 1 compares the approaches by formulation, algorithm, cost, solution quality and relevance to the identified gap.

Table 1. Comparison of approaches relevant to the identified gap

Work	Formulation / algorithm	Computational cost	Solution quality	Relevance to this work
Grothey & McKinnon [1] (selected paper) and our IA-1 code	qq-formulation; trust-region filter SLP + random multistart	low per run (LPs only); many restarts needed	best time-to-quality among tested solvers; 4–20% Good on hard instances	baseline; restart strategy is our target
Baron / spatial B&B (in [1]); Sass et al. [3]	convex relaxations + branch-and-bound; [3] grows the data set during B&B	very high (hours)	global bounds, slow to find good feasible points on af-series	motivates a cheap bound; our diet LB is a fast relaxation
Gill & Zhang [4]	shifted primal-dual interior point + projected search	Newton systems per iteration	robust local method	IPOpt-type methods were slower than SLP in [1]
Muehlebach & Jordan [5]	accelerated first-order method with velocity constraints	cheap, many iterations	stationary points	local only; does not choose basins
di Serafino et al. [6]	stationarity measures, active-set methods for polyhedral constraints	low	better active-set decisions near a solution	our LP sub-problems are polyhedral; supports LP polish
Terzi et al. [2]	matheuristics (metaheuristic + exact sub-solver) vs metaheuristics	moderate	matheuristics win	supports 'heuristic outer loop + exact LP inner solve'
Haverly [8]; Audet et al. [9]	recursion / alternating LPs (ALT); variable neighbourhood search	low	good local solutions on small pooling instances	basis of our product-LP / bin-LP alternation
Lourenço et al. [13];	iterated local search; k-means++	low	standard frameworks	outer loop and initial bins of

Work	Formulation / algorithm	Computational cost	Solution quality	Relevance to this work
Arthur & Vassilvitskii [14]	seeding			RG-ILS-SLP

3. Research Gap & Team Contribution

The gap was identified from three sources: limitations reported by the authors, our IA-1 implementation experience, and IA-1 experimental results compared with the paper.

3.1 Identified Research Gap

Limitation in the literature. The SLP results in [1] rest on random multistart. Every run is independent, and nothing learned in earlier runs is reused. Tables 2 and 4 of [1] show only 4–20% Good runs on af-3, af-4, af-6, af-6-fs and af-7, so ETiGood reaches 6.5–125 s although one run takes 1.3–6.5 s. The authors also note that SLP can stop at a local minimum of the feasibility problem.

Evidence from IA-1. The IA-1 report validated af-1 only (20/20 runs feasible and Good). Our IA-1 notebook also ran the code on seven more instances with the same seed (12345); for af-2, af-6-f and af-7b-fs we ran the same code and seed in IA-2. Table 2 shows the result: af-2, af-3 and af-6-fs never reached a Good solution, af-6-f and af-7 only once, and af-4, af-6-f and af-7 ended infeasible in 25–40% of the runs (stuck in restoration with violation $\approx 1.3 \times 10^{-3}$ on af-4). Re-running that code in IA-2 reproduces the notebook numbers exactly (only Ti differs with the machine).

Table 2. Evidence for the gap: published SLP-CPLEX [1] vs our IA-1 code (IA-1 notebook, 20 runs, seed 12345)

Instance	Paper %Feas	Paper %Good	Paper ETiGood (s)	IA-1 %Feas	IA-1 %Good	IA-1 It
af-1	100.0	79.0	0.6	100	100	72.5
af-2	100.0	19.4	6.7	95	0	114.8
af-3	99.2	20.0	6.5	100	0	48.2
af-4	98.5	19.8	8.6	60	25	77.3
af-5	99.8	100.0	1.5	100	100	139.2
af-5-fs	100.0	99.6	1.7	90	90	250.8
af-6-f	78.4	4.0	112.5	65	5	159.4
af-6-fs	95.0	5.2	125.0	90	0	199.1
af-7	99.0	20.4	22.5	75	5	188.0
af-7b	98.8	71.8	17.1	95	70	137.0
af-7b-fs	99.0	52.6	27.9	100	100	114.8

Gap: random-multistart SLP spends most of its time in poor or infeasible basins. The structure of the problem is not used to choose better starts or to repair end points, even though the problem becomes linear once either the bins or the bin shares are fixed.

3.2 Problem Statement

Given a fixed-demand pooling instance in qq-form, find a strategy around the unchanged IA-1 filter-SLP that (i) increases %Feas, (ii) increases %Good on the hard instances and therefore (iii) reduces the expected time to a Good solution. The optimisation model must not change, and the LP solver stays open-source.

3.3 Team's Proposed Contribution

We do **not** change the objective, the constraints or the SLP algorithm of IA-1. We change what happens *around* the SLP: how starting points are produced, how SLP end points are finished, and how successive runs are linked. This should help because the hard decision is the bin compositions. Once the bins are fixed, the rest is an LP that can be solved exactly, and the diet relaxation tells us which recipe each product would like a bin to have.

Table 3. What is retained, modified and newly introduced

Component	Baseline (paper [1] / our IA-1 code)	Proposed (IA-2)	Status
Model	qq-formulation (5a)–(5h); IA-1 qq_model.py	unchanged (same file)	Retained
Local solver	filter trust-region SLP; IA-1 slp.py (HiGHS)	unchanged (same file, same parameters)	Retained
Starting points	uniform random proportions (QQModel.random_start)	recipe-clustering start + LP repair	Modified (initialisation)
Restarts	independent random multistart	regret-guided ILS from the incumbent; 4 candidates ranked by 10-iteration SLP probes; fresh recipe start after 6 non-improving runs	New (search strategy / hybridisation)
End point	SLP point as returned	LP polish: product LP, then bin LP	New
Bound	none (Baron in [1], hours)	diet-relaxation lower bound in milliseconds	New (auxiliary formulation)
Experiment	20 runs, af-1 (IA-1 report)	all 11 instances × 3 seeds × 20 runs, tests, ablation	Extended

4. Proposed Methodology

The method keeps the original problem and solver and adds a layer of LP sub-problems around them. Section 4.1 gives the original model and the new auxiliary formulations, and Section 4.2 gives the algorithm.

4.1 Proposed Optimization Formulation

Sets and parameters. $i \in I$ raw materials (af files list straight copies separately), $m \in M$ bins, $p \in P$ products, $n \in N$ nutrients; r_{in} nutrient content, c_i unit cost, t_p demand, $[\underline{d}_{pn}, \bar{d}_{pn}]$ product specs, $[\underline{v}_{mn}, \bar{v}_{mn}]$ optional bin specs, bounds on λ and μ (a zero upper bound means the link does not exist). **Variables:** λ_{mi}, μ_{pi}, μ_{pm}, v_{mn}, d_{pn}, c^M_m, c^D_p.

(a) Original problem — qq-formulation (retained unchanged, IA-1 qq_model.py).

$$\min \ \Sigma_p \, t_p \, c^D_p \tag{5a}$$

$$v_{mn} = \Sigma_i \, \lambda_{mi} \, r_{in} \ , \quad c^M_m = \Sigma_i \, \lambda_{mi} \, c_i \qquad \forall \ m, n \tag{5b,f}$$

$$d_{pn} = \Sigma_i \, \mu_{pi} \, r_{in} + \Sigma_m \, \mu_{pm} \, v_{mn} \ , \quad c^D_p = \Sigma_i \, \mu_{pi} \, c_i + \Sigma_m \, \mu_{pm} \, c^M_m \quad \text{(bilinear)} \tag{5c,g}$$

$$\Sigma_i \, \lambda_{mi} = 1 \ \ \forall \ m \ ; \qquad \Sigma_i \, \mu_{pi} + \Sigma_m \, \mu_{pm} = 1 \ \ \forall \ p \tag{5d,e}$$

$$\underline{d}_{pn} \le d_{pn} \le \bar{d}_{pn} \ ; \quad \underline{v}_{mn} \le v_{mn} \le \bar{v}_{mn} \ ; \quad \text{bounds on } \lambda, \mu \tag{5h}$$

In IA-1 the price is attribute $n = N+1$ ($r_{i,N+1} = c_i$), so (5b,f) and (5c,g) are one family: $V_{mq} = \Sigma \, \lambda_{mi} R_{iq}$ and $D_{pq} = \Sigma \, \mu_{pi} R_{iq} + \Sigma \, \mu_{pm} V_{mq}$. SLP linearises the bilinear term at x^k ($\partial/\partial\mu_{pm} = -V_{mq}$, $\partial/\partial V_{mq} = -\mu_{pm}$) and solves $\min c^T x$ subject to the linearised rows and $|x - x^k| \le \rho \cdot w$. This model and solver are **unchanged** in IA-2.

(b) Diet relaxation (new) — ideal recipe and lower bound. For every product p, sharing of bins is relaxed: the product may draw any bin-eligible raw material directly at bin cost.

$$c^*_p = \min_{w \ge 0} \Sigma_i \, c_i \, w_{pi} \quad \text{s.t.} \ \ \Sigma_i \, w_{pi} = 1, \ \ \underline{d}_{pn} \le \Sigma_i \, w_{pi} \, r_{in} \le \bar{d}_{pn}, \ \ w_{pi} \in [\underline{\mu}_{pi}, \bar{\mu}_{pi}] \text{ for straights}, \ \ \Sigma_{i \in B(p)} \, w_{pi} \le \Sigma_m \, \bar{\mu}_{pm} \tag{D1}$$

$$LB = \Sigma_p \, t_p \, c^*_p \ \le \ f^*_{qq} \ ; \qquad \text{ideal bin recipe} \ \ q_p = w^B_p \ / \ \Sigma_i \, w^B_{pi} \tag{D2}$$

B(p) is the set of raw materials that can reach product p through some bin. Any qq-feasible point gives product p the effective recipe $\Sigma_m \mu_{pm}\lambda_{m\cdot} + \mu_{p\cdot}$, which is feasible for (D1). Hence (D2) is a valid lower bound. (D1) ignores bin sharing, bin bounds and bin specs.

(c) Product LP with bins fixed (new). For fixed $\bar{\lambda}$, v and c^M are constants, so (5c) and (5g) become linear:

$$\min_{\mu} \Sigma_p\, t_p\, (\Sigma_i\, \mu_{pi}\, c_i + \Sigma_m\, \mu_{pm}\, \bar{c}^M{}_m) \;\; \text{s.t.} \;\; \underline{d}_{pn} \le \Sigma_i\, \mu_{pi}\, r_{in} + \Sigma_m\, \mu_{pm}\, \bar{v}_{mn} \le \bar{d}_{pn}, \;\; (5e), \text{ bounds} \qquad \text{(PLP)}$$

(d) Bin LP with bin shares fixed (new). For fixed μ^B, substituting (5b) into (5c) gives d linear in (λ, μ^S):

$$\min_{\lambda,\mu S} \Sigma_p\, t_p\, (\Sigma_i\, \mu_{pi}\, c_i + \Sigma_m\, \bar{\mu}_{pm}\, \Sigma_i\, \lambda_{mi}\, c_i) \;\; \text{s.t.} \;\; \underline{d}_{pn} \le \Sigma_i\, \mu_{pi}\, r_{in} + \Sigma_m\, \bar{\mu}_{pm}\, \Sigma_i\, \lambda_{mi}\, r_{in} \le \bar{d}_{pn}, \;\; \underline{v} \le \lambda r \le \bar{v}, \; (5d), \; (5e) \qquad \text{(BLP)}$$

(e) Recipe regret and guided perturbation (new).

$$\text{regret}_p = t_p \cdot \max(0,\, c^D{}_p - c^*{}_p)\;; \qquad P(p) \propto \text{regret}_p\,, \quad m \sim \text{uniform} \qquad \text{(R1)}$$

$$\bar{\lambda}_{m\cdot} = \text{Proj}(\; (1-\alpha)\; q_p + \alpha\; \lambda_{m\cdot}\;), \;\; \alpha \sim U(0,\, 0.5); \quad \text{all bins} \times \exp(0.05\; \xi), \;\; \xi \sim N(0,\, 1) \qquad \text{(R2)}$$

Elastic versions of (PLP) and (BLP) add penalised slacks $e \ge 0$ to the quality rows (normalised cost + $10^4\, \Sigma e$). They are used when the fixed part makes the LP infeasible. Proj(·) is an ℓ_1 projection LP onto the bin's simplex, its λ bounds and its nutrient bounds. **The mathematical change in short:** the nonconvex problem (5a)–(5h) is unchanged. We add the convex relaxation (D1)–(D2) and the two exact linear restrictions (PLP) and (BLP) of the bilinear rows (5c,g), and use their solutions to generate, repair and polish SLP points. All LPs are built on the IA-1 variable indexing (lam_var, mur_var, mub_var, V_var, D_var), so every point can be passed straight to FilterSLP.solve().

4.2 Proposed Algorithm / Method

Step 1. **[New] Diet relaxation.** Solve (D1) for every product to get the ideal recipes q_p, unit costs c^*_p and the bound LB. This happens once per instance.

Step 2. **[Modified] Recipe-clustering start.** Mix each q_p with 30% Dirichlet noise, run weighted k-means++ (weights t_p × bin share) with M clusters, and project the centres onto the bin constraints.

Step 3. **[New] LP repair.** Solve the elastic (PLP) with these bins. While the violation is > 10^{-7}, alternate the elastic (BLP) and (PLP), at most 6 rounds.

Step 4. **[Retained] Filter-SLP.** Run FilterSLP.solve() of IA-1 from this point, unchanged.

Step 5. **[New] LP polish.** Fix the final bins and solve (PLP), then fix the bin shares and solve (BLP), for at most 10 rounds. Keep a point only if it is feasible and cheaper. If the SLP ended infeasible, LP repair with its bins is tried first. Update the incumbent.

Step 6. **[New] Restart rule.** Runs 1–3, and any run after 6 ILS runs without improving the incumbent, go back to Step 2 (diversification).

Step 7. **[New] Regret-guided ILS run.** Otherwise build 4 candidates from the incumbent with (R1)–(R2), repair each (Step 3), probe each with a 10-iteration SLP, continue the full SLP (Step 4) from the best probe (feasible first, then lowest cost), and go to Step 5.

Each pass through Steps 2–7 counts as one *run*, including the probes. Run-based metrics (%Feas, %Good, Ti, It) are therefore comparable with the baseline, whose run is one random start followed by SLP. The probe step came from implementation experience: from an af-3 incumbent stuck 0.96% above the best known value, we tried all 56 single re-seeding moves (8 highest-regret products × 7 bins). Only 3 of them led the full SLP to a Good solution. The LP cost of the repaired point did not identify them, but a 10- or 25-iteration SLP probe ranked an escaping move first in both cases.

4.3 Methodology Flowchart

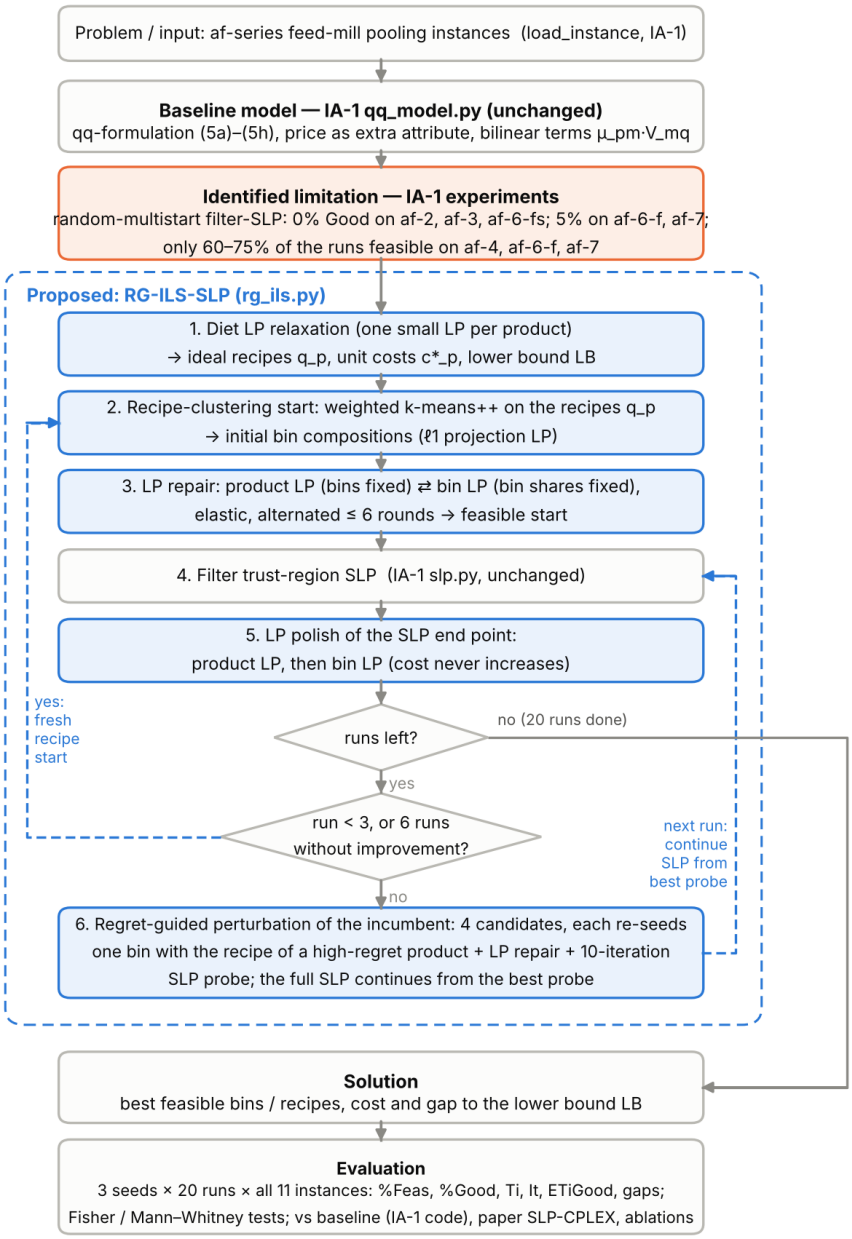


Figure 1. Methodology flowchart: IA-1 model and solver (grey, unchanged), identified limitation (orange) and the proposed RG-ILS-SLP loop (blue = new).

4.4 Algorithm / Pseudocode

Algorithm 1 — RG-ILS-SLP (proposed).

```
Input : instance, number of runs K = 20, seed
Output: best feasible point x*, lower bound LB
1  (q, c*, LB) <- DietRelaxation()                # P small LPs (once)
2  x* <- None ; stall <- 0
3  for k = 0 .. K-1 do
4      if k < 3 or stall >= 6:                     # diversification
```

```
5          x0 <- Repair(Project(WeightedKMeans(q + noise, M))) ; stall <- 0
6      else:                                        # regret-guided ILS
7          for j = 1..4:  p ~ regret(x*), m ~ U{1..M}, alpha ~ U(0, 0.5)
8              Lam_j <- Project(Lam(x*) with row m <- (1-alpha) q_p + alpha Lam_m(x*))
9              y_j  <- FilterSLP_10it(Repair(Lam_j))  # short probe
10         x0 <- best y_j  (feasible first, then cost)
11     x <- Polish(FilterSLP(x0))                   # IA-1 SLP unchanged + LP polish
12     if feasible(x) and f(x) < f(x*): x* <- x ; stall <- 0   else if ILS run: stall += 1
13 end for
14 return x*, LB
```

Algorithm 2 — Filter-SLP core (IA-1 slp.py, retained unchanged).

```
FilterSLP.solve(x)   (IA-1 slp.py, unchanged): rho <- 0.5 ; filter F <- {}
for k = 1..400:
   LP: min c'y  s.t. linearised rows at x, lb <= y <= ub, |y - x| <= rho*width    (HiGHS, warm basis)
   if LP infeasible: block restoration (exact LPs, mu or (lam,V) fixed), else elastic l1 LP
   if h(x) <= 1e-6 and (step <= 1e-9 or dq <= 1e-9(1+|f|)): stop (converged)
   accept if sufficient decrease (sigma = 0.1) and not dominated by F (beta = 0.99, gamma = 0.01)
   accept: x <- y ; rho <- min(1, 2 rho) at the boundary;   reject: rho <- 0.5 min(rho, step)
until rho < 1e-9
```

5. Implementation & Experimental Evaluation

Both methods use the same IA-1 files: the parser and qq-model (qq_model.py), the filter-SLP (slp.py) and the experiment runner (run_experiments.py, extended with --method). The only new algorithmic file is rg_ils.py. The baseline and proposed runs differ only in the starting points, the end-point polish and the restart strategy, so the comparison isolates the team's contribution.

5.1 Implementation Environment

Table 4. Implementation environment

Item	Details
Programming Language	Python 3.13
Libraries / Tools	NumPy, SciPy 1.18 (bundled HiGHS LP solver: warm-started dual simplex in slp.py, linprog in rg_ils.py), pandas, matplotlib, openpyxl
Hardware / Runtime	Linux cloud container, 2 vCPU, 7 GB RAM; single-threaded runs, two experiment queues in parallel. IA-1 notebook: Windows laptop (Anaconda). Paper [1]: Xeon E5-2670 2.6 GHz, CPLEX
Dataset	af-1, af-3, af-4, af-5, af-5-fs, af-6-fs, af-7, af-7b (Grothey & McKinnon [10]); the same files as IA-1
Baseline Algorithm	IA-1 code unchanged: QQModel.random_start + FilterSLP (ρ0 = 0.5, ρmax = 1, β = 0.99, γ = 0.01, σ = 0.1, ≤ 400 it.)
Proposed Algorithm	RG-ILS-SLP (rg_ils.py) around the same FilterSLP
Parameter Settings	Dirichlet noise 0.3; ≤ 6 repair rounds; elastic penalty 1e4; ≤ 10 polish rounds; 3 initial recipe starts; patience 6; 4 candidates × 10-iteration probes; α ∼ U(0, 0.5); bin noise 0.05
Stopping Criteria	SLP as in IA-1. Feasible: max scaled violation ≤ 1e-6. Good: feasible and ≤ 1.002 × best known. 20 runs per seed and instance

5.2 Dataset / Problem Instances

The instances come from industrial animal-feed practice and were published with [1] [10]. They are AMPL data files with nutrient contents, product specs, tonnages, raw-material costs, bounds on bin and product proportions and, in some files, bin nutrient bounds. We use all eleven instances; the paper's af-6 is the file af-6-f.txt. The IA-1 parser is reused unchanged, including its handling of the swapped λ/μ names in the files.

Table 5. Test instances (sizes of the qq-model built by the IA-1 code) and lower bounds

Instance	N	I	M	P	n (vars)	m (cons)	Best known [1]	Diet LB (ours)	Baron LB [1]	Diet gap %
af-1	8	32	6	25	925	310	296.45	295.93	295.93	0.17
af-2	9	37	7	30	871	407	239,334.02	235,742.77	235,751.11	1.50
af-3	11	33	7	27	819	442	122,257.45	118,915.82	120,498.73	2.73
af-4	14	32	6	32	973	608	186,097.11	170,332.92	183,620.39	8.47
af-5	17	34	7	18	816	475	124,003.48	110,795.29	123,243.60	10.65
af-5-fs	17	58	7	18	1266	475	119,016.92	106,716.11	118,763.29	10.34
af-6-f	17	35	7	40	1402	893	3,074.23	2,743.67	2,937.68	10.75
af-6-fs	17	62	7	40	2555	893	2,921.30	2,699.07	2,881.41	7.61
af-7	14	43	7	50	1683	912	151,381.81	126,643.66	145,928.01	16.34
af-7b	14	43	14	50	2320	1024	147,285.00	126,643.66	145,928.01	14.01
af-7b-fs	14	70	14	50	3788	1024	130,290.06	99,754.48	130,147.66	23.44

Problem sizes (n, m) are those of the IA-1 qq-model. The last three columns compare our diet-relaxation bound with Baron's bound from [1]. On af-1 they agree to five digits; on the instances with restricted bins the diet bound is weaker (1.5–23% below the best known value), because it ignores bin sharing. There is no train/test split, since each instance is an independent optimisation problem.

5.3 Experimental Design

Each method is run on each instance with 3 seeds (12345, the IA-1 seed, then 2024 and 7) × 20 runs. In total that is 1,320 runs, plus 440 ablation runs with seed 12345. The metrics follow [1]: %Feas, %Good, Ti (s per run), It (SLP iterations per run) and ETiGood = 100·Ti/%Good, pooled over the 60 runs. We add the per-seed spread, the median gap of feasible runs, the time to the first Good run, a one-sided Fisher exact test on Good and feasible counts, and a one-sided Mann–Whitney U test on final gaps (infeasible runs count as worst).

Table 6. Experimental design

Experiment	Method	Parameters	Metric(s)	Purpose
E1 Baseline	IA-1 code, random multistart	3 seeds × 20 runs × 11 instances	%Feas, %Good, Ti, It, ETiGood	reference; seed 12345 = IA-1 notebook
E2 Proposed	RG-ILS-SLP	3 seeds × 20 runs × 11 instances	same + gaps, tests	evaluate the contribution
E3 Ablation A	random start + SLP + LP polish	seed 12345, 20 runs × 11	%Feas, %Good	effect of polish alone
E4 Ablation B	recipe start + repair + SLP + polish, independent runs	seed 12345, 20 runs × 11	%Feas, %Good	effect of the start without ILS
E5 External	published SLP-CPLEX [1]; IA-1 notebook	500 runs [1]; 20 runs (IA-1)	%Good, ETiGood	position against the literature

5.4 Implementation Evidence

Before the experiments we checked the implementation. With seed 12345 the baseline reproduces the IA-1 notebook exactly on all eight instances that the notebook ran (same %Feas, %Good, It and best objective, e.g. af-1: 100%, 72.5 iterations, 296.44604). Points assembled from LP solutions satisfy every qq-equation to ~10^{-14}. The af-1 diet bound (295.93153) equals Baron's bound. Re-running the final code (run_all.py or the notebook IA2_Demo_Colab.ipynb) reproduces the stored per-run costs exactly on 10 of 11 instances; on af-7b-fs, the largest, the IA-1 limit of 3 s per LP is sometimes hit on a busier machine, so a few runs differ slightly (≤ 0.015% in cost in our test). Listings 1–3 show key code and console output. The full code, logs and per-run CSV files are in the submission folder.

Listing 1. Product LP with the bins fixed, built on the IA-1 variable indexing (rg_ils.py, abridged).

```
def product_lp(mod, Lam, elastic=False):      # bins fixed -> LP in mu (all products, one HiGHS call)
    V = Lam @ mod.R                            # bin attributes incl. price column (IA-1 scaling)
    for k in range(mod.P):                     # block k: composition = mur R + mub V
        comp = vstack([mod.R[js], V[jb]])      # js / jb = allowed straights / bins (IA-1 index
arrays)
        rows: lb[D_var[k,:N]] <= comp[:, :N].T z <= ub[D_var[k,:N]],  sum z = 1
    z <- linprog(cost, block-diagonal rows, bounds from mod.lb / mod.ub)
    return assemble(mod, Lam, Mur, Mub)        # IA-1 vector x with V, D computed exactly
```

Listing 2. Regret-guided perturbation of the incumbent (rg_ils.py).

```
def recipe_guided_perturb(mod, x, diet, rng):
    Lam, _, Mub = proportions(mod, x)
    reg = regret(mod, x, diet) + 1e-9          # t_p * max(0, cD_p - c*_p)
    m = rng.choice(mod.M, 1, replace=False, p=uniform)
    p = rng.choice(mod.P, p=reg / reg.sum())   # product that pays most for bin sharing
    a = rng.uniform(0.0, 0.5)
    target = Lam.copy(); target[m] = (1 - a) * diet['recipes_bin'][p] + a * Lam[m]
    target *= exp(0.05 * rng.standard_normal(target.shape))
    return project_bins(mod, target / target.sum(1))      # l1-projection LP
```

Listing 3. Console output (excerpt) of the proposed method on af-4, seed 12345 (rc = recipe start, ils = probed ILS run).

```
$ python run_experiments.py --method proposed --instances af-4 --runs 20 --seed 12345
[af-4] run  0 rc   f=186506.98595 feas=1 it=28 t=1.17s  incumbent=186506.98595
[af-4] run  1 rc   f=186871.92240 feas=1 it=400 t=3.41s  incumbent=186506.98595
[af-4] run  2 rc   f=186446.94673 feas=1 it=126 t=2.09s  incumbent=186446.94673
[af-4] run  3 ils  f=193193.00277 feas=0 it=440 t=8.50s  incumbent=186446.94673
[af-4] run  4 ils  f=186446.94674 feas=1 it=101 t=3.99s  incumbent=186446.94673
[af-4] run  5 ils  f=186446.94673 feas=1 it=138 t=3.24s  incumbent=186446.94673
[af-4] run  6 ils  f=192311.86971 feas=0 it=440 t=9.06s  incumbent=186446.94673
[af-4] run  7 ils  f=186446.94673 feas=1 it=128 t=3.53s  incumbent=186446.94673
[af-4] run  8 ils  f=186494.40060 feas=1 it=59 t=3.50s  incumbent=186446.94673
[af-4] run  9 rc   f=198875.98146 feas=0 it=400 t=7.72s  incumbent=186446.94673
[af-4] run 10 ils  f=186446.94674 feas=1 it=81 t=3.86s  incumbent=186446.94673
[af-4] run 11 ils  f=186466.86021 feas=1 it=42 t=3.58s  incumbent=186446.94673
  ...
```

6. Results, Comparison & Discussion

In this section **IA-1** denotes the baseline, our unchanged IA-1 code (random-multistart filter-SLP), and **IA-2** the proposed RG-ILS-SLP; both are run with the same instances, seeds and machine. All numbers below come from the per-run CSV files in 06_Results. Paper values are SLP-CPLEX from Tables 2, 4 and 5 of [1] (500

runs; the paper's af-6 is our af-6-f). IA-1 values are from our IA-1 notebook (20 runs, seed 12345); for af-2, af-6-f and af-7b-fs, which the notebook did not run, they come from the same code and seed run in IA-2.

6.1 Results of IA-1 (Baseline Method)

The baseline solves the easy instances reliably: af-1, af-5 and af-7b-fs are Good in 100.0%, 98.3% and 95.0% of the runs. The hard instances confirm the gap: %Good is 1.7% on af-2, 1.7% on af-3, 20.0% on af-4, 1.7% on af-6-f, 3.3% on af-6-fs and 10.0% on af-7. Only 68.3%, 73.3% and 70.0% of the af-4, af-6-f and af-7 runs end feasible, and on af-5-fs 20% of the runs fail because of infeasibility, not quality. Once the baseline reaches a good basin it finds very good points: the best known value on af-4 and af-7b-fs and 0.0015% above it on af-7.

Table 7. IA-1 results (baseline: our IA-1 code, random-multistart filter-SLP), 3 seeds × 20 runs per instance

Instance	Ti (s)	%Feas	%Good (mean ± sd over seeds)	It	ETiGood (s)	Best f	Best gap %
af-1	0.41	100.0	100.0 ± 0.0	80.4	0.4	296.45	0.000
af-2	1.00	98.3	1.7 ± 2.9	110.4	60.1	239,804.20	0.196
af-3	1.01	96.7	1.7 ± 2.9	73.4	60.9	122,316.17	0.048
af-4	3.37	68.3	20.0 ± 5.0	85.8	16.9	186,097.10	0.000
af-5	1.28	98.3	98.3 ± 2.9	118.2	1.3	124,007.33	0.003
af-5-fs	3.81	80.0	80.0 ± 10.0	254.6	4.8	119,021.91	0.004
af-6-f	6.16	73.3	1.7 ± 2.9	162.2	369.6	3,074.74	0.016
af-6-fs	12.28	90.0	3.3 ± 5.8	196.0	368.3	2,925.29	0.136
af-7	7.26	70.0	10.0 ± 5.0	206.8	72.6	151,384.01	0.001
af-7b	10.70	95.0	68.3 ± 2.9	143.1	15.7	147,318.57	0.023
af-7b-fs	58.52	96.7	95.0 ± 5.0	120.2	61.6	130,289.36	-0.001

6.2 Results of IA-2 (Proposed Method)

The proposed method ends feasible in 96.1% of the runs; every run is feasible on 8 of 11 instances (af-2: 96.7%, af-4: 86.7%, af-6-f: 73.3%). On the hard instances %Good is 68.3% (af-2), 31.7% (af-3), 58.3% (af-4), 26.7% (af-6-fs) and 18.3% (af-7), but 0.0% on af-6-f, where its best run is 0.50% above the best known value. It finds the best known value on af-2, 122,267.31 on af-3 (+0.008%) and 2,922.56 on af-6-fs (+0.04%), all better than any baseline run. A proposed run costs more time on average (LP repair, four probes, polish), but the ILS runs need fewer SLP iterations on the larger instances (af-7: 120 vs 207).

Table 8. IA-2 results (proposed RG-ILS-SLP), 3 seeds × 20 runs per instance

Instance	Ti (s)	%Feas	%Good (mean ± sd over seeds)	It	ETiGood (s)	Best f	Best gap %
af-1	0.55	100.0	100.0 ± 0.0	55.1	0.6	296.45	0.000
af-2	2.15	96.7	68.3 ± 2.9	113.9	3.1	239,330.51	-0.001
af-3	2.22	100.0	31.7 ± 15.3	81.2	7.0	122,267.31	0.008
af-4	6.27	86.7	58.3 ± 2.9	126.6	10.7	186,446.95	0.188
af-5	2.73	100.0	98.3 ± 2.9	120.5	2.8	124,006.41	0.002
af-5-fs	5.31	100.0	98.3 ± 2.9	230.0	5.4	119,021.91	0.004
af-6-f	10.15	73.3	0.0 ± 0.0	194.8	∞	3,089.51	0.497
af-6-fs	13.58	100.0	26.7 ± 29.3	155.4	50.9	2,922.56	0.043

Instance	Ti (s)	%Feas	%Good (mean ± sd over seeds)	It	ETiGood (s)	Best f	Best gap %
af-7	7.95	100.0	18.3 ± 7.6	119.7	43.4	151,391.44	0.006
af-7b	9.95	100.0	81.7 ± 2.9	109.5	12.2	147,305.30	0.014
af-7b-fs	65.30	100.0	100.0 ± 0.0	104.1	65.3	130,290.64	0.000

6.3 Comparison: IA-1 (Baseline) vs IA-2 (Proposed)

Table 9 sets IA-1 and IA-2 side by side: its first six rows list what IA-2 changes, the remaining rows what it achieves. Averaged over all 11 instances, IA-2 improves %Feas over IA-1 by +8.2 points and %Good by +18.3 points. On the 6 hard instances %Good rises from 6.4% to 33.9%, and the expected time to a Good solution is 5.0× shorter (geometric mean over af-2, af-3, af-4, af-6-fs and af-7; on af-6-f the proposed method has no Good run). Over all instances where both methods find a Good run the speed-up is 2.0×, lower because the easy instances become slower.

Table 9. IA-1 vs IA-2: what changed and what it achieved (results pooled over 11 instances × 3 seeds × 20 runs)

Aspect / metric	IA-1 (baseline)	IA-2 (proposed)	Change	Observation
Model and SLP solver	qq_model.py + slp.py	same files, unchanged	retained	only what happens around the SLP changes
Starting points	uniform random proportions	recipe clustering (diet LP + k-means++) + LP repair	modified	cost-aware, near-feasible starts
Restarts	independent random multistart	regret-guided ILS from the incumbent; 4 moves ranked by 10-iteration SLP probes	new	later runs reuse the best solution found
SLP end point	returned as is	LP polish (product LP, then bin LP)	new	exact feasibility, cost never increases
Lower bound	none	diet LP (af-1: equal to Baron's bound)	new	quality certificate in milliseconds
Evaluation	IA-1 report: af-1, 20 runs	11 instances × 3 seeds × 20 runs, tests, ablation	extended	same protocol for both methods
%Feas (mean of 11)	87.9	96.1	+8.2 pts	all runs feasible on 8/11 instances (baseline 1/11)
%Good (mean of 11)	43.6	62.0	+18.3 pts	significant (Fisher, $p < 0.01$) on af-2, af-3, af-4, af-5-fs, af-6-fs
%Good, 6 hard instances	6.4	33.9	+27.5 pts	paper SLP-CPLEX: 14.8%
Ti, s per run (mean)	9.62	11.47	+19%	LP repair, probes and polish cost time per run
SLP iterations per run (mean)	141.0	128.2	-9%	incl. the 4 × 10 probe iterations of ILS runs
ETiGood speed-up, hard inst. (geo-mean)	1.0×	5.0×	5.0× faster	over af-2, af-3, af-4, af-6-fs, af-7 (af-6-f: proposed has no Good run)
ETiGood speed-up, all with Good runs (geo-mean)	1.0×	2.0×	2.0×	slower on af-1, af-5, af-5-fs, af-6-f, af-7b-fs
Median gap of feasible runs, % (mean of 11)	0.432	0.311	-0.121 pts	Mann–Whitney $p < 0.01$ on 9 of 11 instances

Table 10. Per instance: published SLP-CPLEX [1] vs IA-1 vs IA-2 (%Good and ETiGood in s; IA-1 and IA-2 pooled over 3 × 20 runs)

Instance	Paper %Good	IA-1 %Good	IA-2 %Good	Paper ETiGood	IA-1 ETiGood	IA-2 ETiGood
af-1	79.0	100.0	100.0	0.6	0.4	0.6
af-2	19.4	1.7	68.3	6.7	60.1	3.1
af-3	20.0	1.7	31.7	6.5	60.9	7.0
af-4	19.8	20.0	58.3	8.6	16.9	10.7
af-5	100.0	98.3	98.3	1.5	1.3	2.8
af-5-fs	99.6	80.0	98.3	1.7	4.8	5.4
af-6-f	4.0	1.7	0.0	112.5	369.6	∞
af-6-fs	5.2	3.3	26.7	125.0	368.3	50.9
af-7	20.4	10.0	18.3	22.5	72.6	43.4
af-7b	71.8	68.3	81.7	17.1	15.7	12.2
af-7b-fs	52.6	95.0	100.0	27.9	61.6	65.3

Table 10 adds the published SLP-CPLEX results. IA-2 has a higher %Good than the paper on 7 of 11 instances (af-1, af-2, af-3, af-4, af-6-fs, af-7b and af-7b-fs). The differences are largest on the instances that motivated the gap: af-2 68.3% vs 19.4%, af-4 58.3% vs 19.8%, af-6-fs 26.7% vs 5.2% and af-3 31.7% vs 20.0%. It is lower on af-6-f (0.0% vs 4.0%) and af-7 (18.3% vs 20.4%), and marginally lower on af-5 and af-5-fs (98.3% vs 100% and 99.6%). ETiGood beats the paper on af-2 (3.1 s vs 6.7 s), af-6-fs (50.9 s vs 125 s), af-7b and af-1, is close on af-3 (7.0 s vs 6.5 s), and is slower elsewhere: Python/HiGHS is slower per LP than C/CPLEX, and the paper used 500 runs on different hardware. The comparison with our own baseline uses identical code, machine and seeds.

6.4 Statistical / Repeated-Run Analysis

Table 11 tests whether IA-2 is better than IA-1 on the pooled 60 runs per method. More Good runs is significant at the 1% level on af-2, af-3, af-4, af-5-fs and af-6-fs, and more feasible runs at the 5% level on af-4, af-5-fs, af-6-fs and af-7. Lower final gaps (Mann–Whitney, 1% level) are significant on 9 of 11 instances; the exceptions are af-1, where both methods are always Good, and af-6-f, where neither is reliable. Table A1 shows the per-seed results: on the hard instances the proposed method beats the baseline's %Good for every seed except af-7 with seed 2024 (10% vs 10%) and af-6-f (0% vs 5%, 0% and 0%). The spread between seeds is large on af-6-fs (5–60%) because a chain either finds the good basin early and stays there, or does not. ILS runs within one seed are not independent, so the tests treat each run as one sample of 'one run of the search' and slightly overstate the effective sample size. The between-seed spread is the more conservative measure.

Table 11. One-sided tests, IA-2 better than IA-1 (60 vs 60 runs per instance)

Instance	Good IA-1	Good IA-2	p Fisher	Feas IA-1	Feas IA-2	p Fisher	Median gap IA-1 %	Median gap IA-2 %	p Mann–Whitney
af-1	60/60	60/60	1.0000	60/60	60/60	1.0000	0.002	0.002	0.0634
af-2	1/60	41/60	< 0.0001	59/60	58/60	0.8782	0.618	0.089	< 0.0001
af-3	1/60	19/60	< 0.0001	58/60	60/60	0.2479	1.065	0.965	< 0.0001
af-4	12/60	35/60	< 0.0001	41/60	52/60	0.0138	0.360	0.199	< 0.0001
af-5	59/60	59/60	0.7521	59/60	60/60	0.5000	0.035	0.023	0.0004
af-5-fs	48/60	59/60	0.0010	48/60	60/60	0.0001	0.011	0.006	0.0002
af-6-f	1/60	0/60	1.0000	44/60	44/60	0.5817	1.005	1.167	0.7554

Instance	Good IA-1	Good IA-2	p Fisher	Feas IA-1	Feas IA-2	p Fisher	Median gap IA-1 %	Median gap IA-2 %	p Mann–Whitney
af-6-fs	2/60	16/60	0.0003	54/60	60/60	0.0137	1.014	0.284	< 0.0001
af-7	6/60	11/60	0.1475	42/60	60/60	< 0.0001	0.936	0.778	0.0020
af-7b	41/60	49/60	0.0696	57/60	60/60	0.1218	0.155	0.111	0.0023
af-7b-fs	57/60	60/60	0.1218	58/60	60/60	0.2479	0.031	0.020	0.0003

6.5 Graphical Results

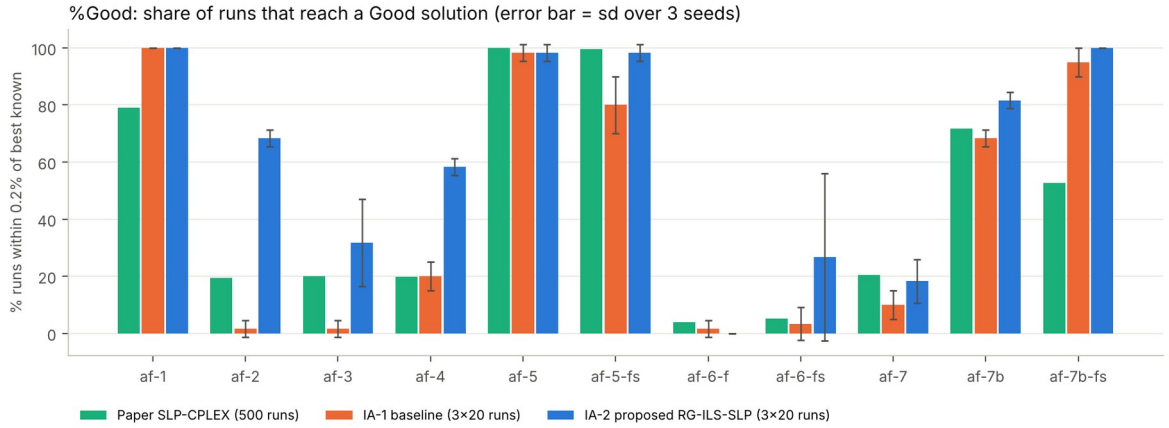


Figure 2. %Good per instance: paper SLP-CPLEX (500 runs), IA-1 (baseline) and IA-2 (proposed) (3 × 20 runs; error bars = sd over seeds).

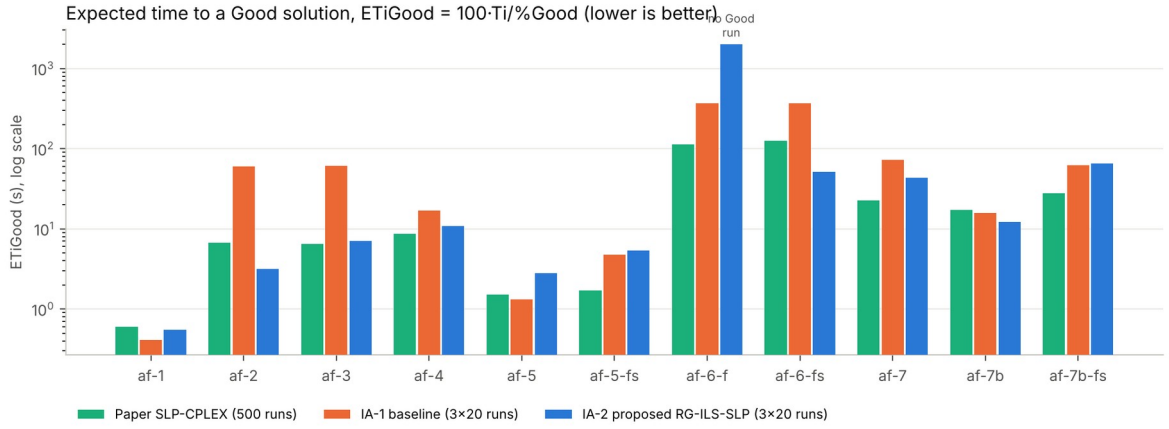


Figure 3. Expected time to a Good solution, paper vs IA-1 vs IA-2 (log scale; bars at the top of the axis mean no Good solution found).

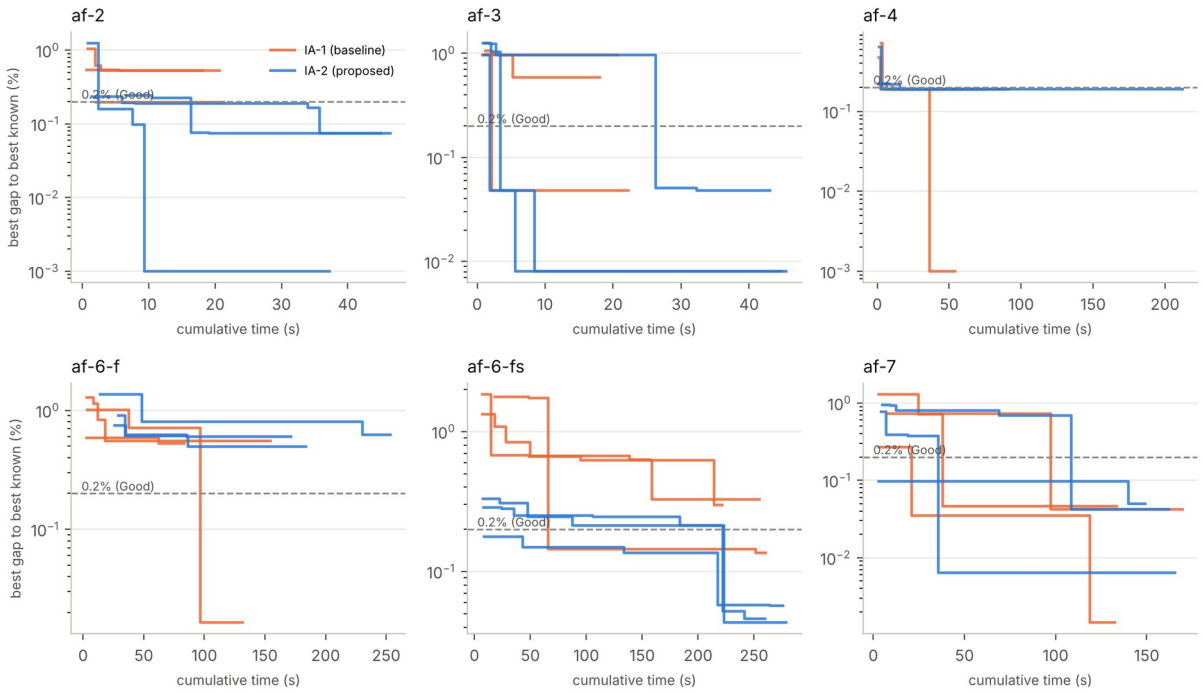


Figure 4. IA-1 vs IA-2: best gap to the best known value against cumulative time on the six hard instances (one line per seed).

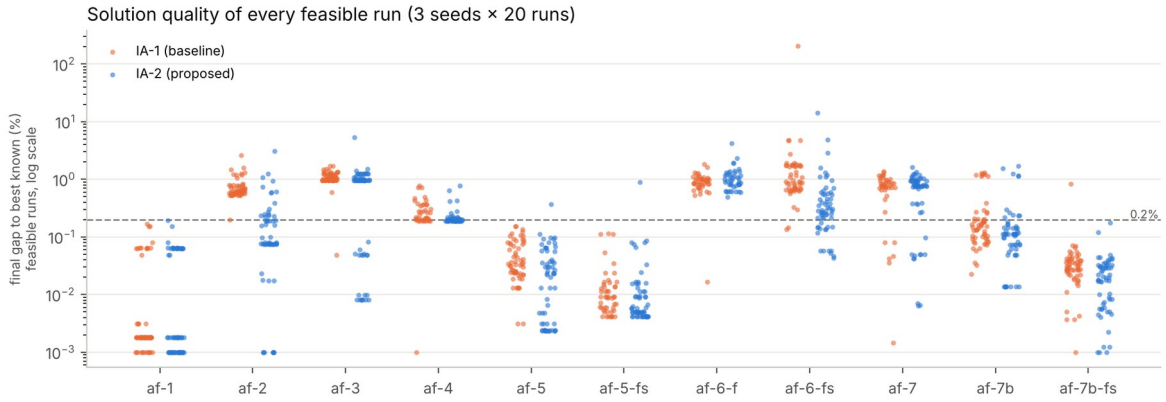


Figure 5. IA-1 vs IA-2: final gap of every feasible run (log scale); dashed line = 0.2% Good threshold.

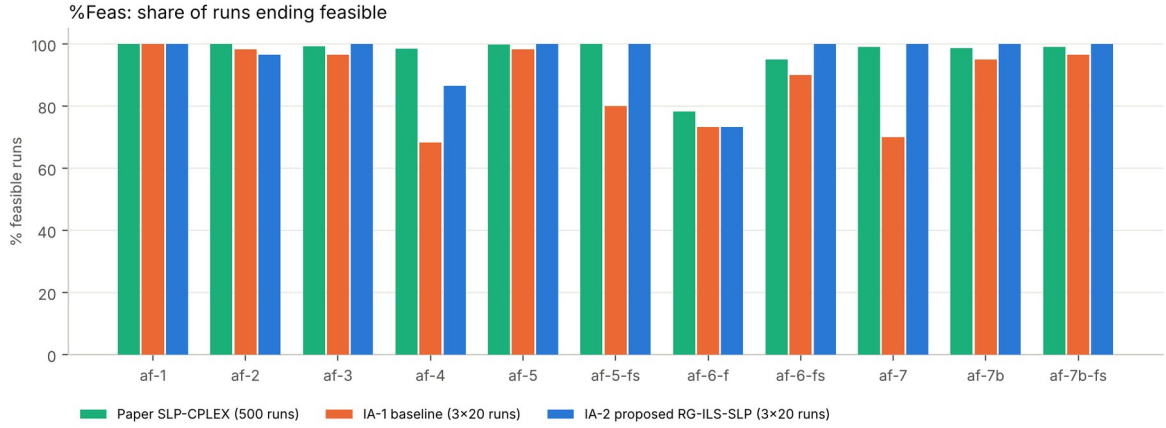


Figure 6. %Feas per instance: paper vs IA-1 vs IA-2.

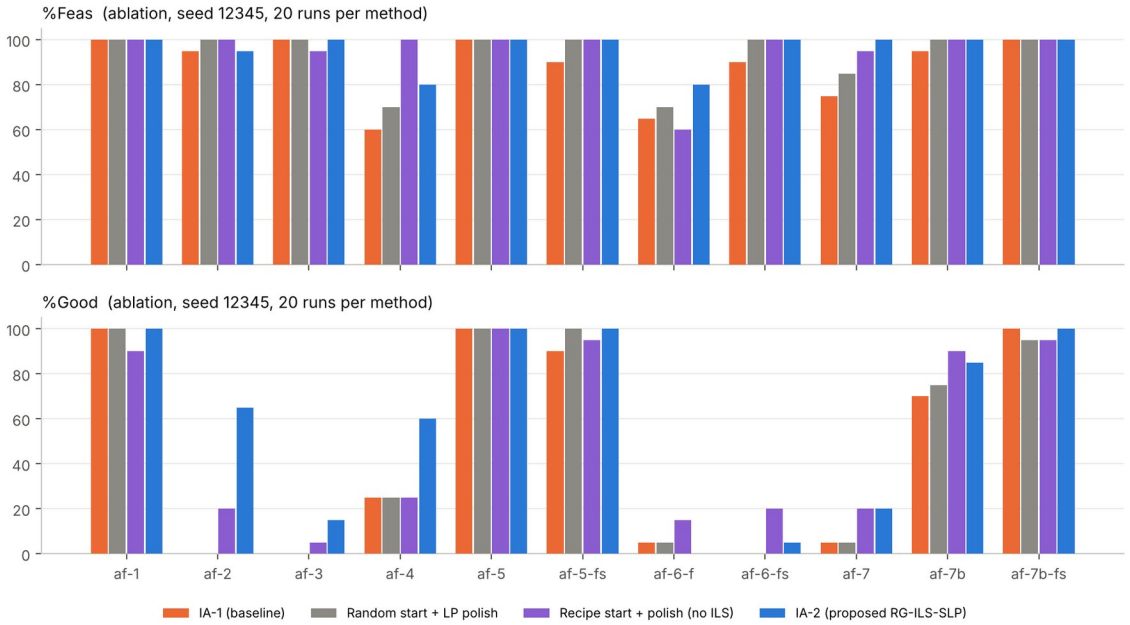


Figure 7. Ablation (seed 12345, 20 runs each): %Feas and %Good of IA-1, the two partial IA-2 variants and the full IA-2 method.

Table 12. Ablation, seed 12345: %Feas / %Good over 20 runs

Instance	IA-1 (baseline)	A: random start + polish	B: recipe start + polish, no ILS	IA-2 full (RG-ILS-SLP)
af-1	100 / 100	100 / 100	100 / 90	100 / 100
af-2	95 / 0	100 / 0	100 / 20	95 / 65
af-3	100 / 0	100 / 0	95 / 5	100 / 15
af-4	60 / 25	70 / 25	100 / 25	80 / 60
af-5	100 / 100	100 / 100	100 / 100	100 / 100
af-5-fs	90 / 90	100 / 100	100 / 95	100 / 100
af-6-f	65 / 5	70 / 5	60 / 15	80 / 0
af-6-fs	90 / 0	100 / 0	100 / 20	100 / 5
af-7	75 / 5	85 / 5	95 / 20	100 / 20
af-7b	95 / 70	100 / 75	100 / 90	100 / 85
af-7b-fs	100 / 100	100 / 95	100 / 95	100 / 100

6.6 Discussion

Does the method close the gap? Mostly yes. The gap was low %Good and failed runs on the hard instances, caused by memory-less random restarts. The proposed method makes almost every run feasible and multiplies %Good on af-2 (×41), af-3 (×19), af-4 (×2.9) and af-6-fs (×8). It does not close the gap on af-6-f (the paper's af-6), where both methods stay about 1% above the best known value. Figure 4 shows how it works on the others: after the first recipe starts the incumbent is already close to 0.2%, and the probed ILS moves search its neighbourhood instead of restarting from scratch.

Which component matters? (Table 12, Figure 7, seed 12345.) The LP polish alone mainly fixes feasibility (af-4 60% → 70%, af-5-fs 90% → 100%, af-6-fs 90% → 100%) and leaves %Good on the hard instances almost unchanged. Independent recipe-clustering starts help most on af-6-fs (20%), af-7 (20%) and af-7b (90%), but not on af-3 (5%) or af-4 (25%). The full ILS adds the large gains on af-2 (20% → 65%), af-3 (15%)

and af-4 (60%). With this seed, recipe starts alone are better on af-6-fs (20% vs 5%) and, notably, on af-6-f (15% vs 0%): there the ILS keeps refining an incumbent that is about 1% above the best known value instead of exploring. The components therefore complement each other: fresh cost-aware starts give diversity, and the ILS gives depth.

Why does it work? In the qq-formulation the hard decisions are the bin compositions. Once they are fixed, the product LP gives the optimal recipes exactly. The diet relaxation shows which product pays most for bin sharing, so re-seeding a bin with that product's ideal recipe is a targeted move rather than a random jump. The short SLP probe tells whether the move leads anywhere before the full SLP is spent on it.

What did not work (implementation experience). Before the final design we tested, on af-3, af-4 and af-7, a smaller initial trust region for the guided starts ($\rho_0 = 0.05$ or 0.1), a trust region on the proportions only, choosing the bin by 1/usage, re-seeding more bins after failures, and ranking candidates by their LP cost. None of these was consistently better, and the small trust region was clearly worse (%Feas fell to 55–80%). Results from a single seed varied by up to 45 points between configurations, which is why the final comparison uses three seeds.

Limitations. (i) af-6-f is not solved: no proposed run reaches 0.2% (baseline 1 of 60, paper 4.0%), and af-7 stays slightly below the paper's %Good. (ii) On af-4 the proposed best is 0.19% above the best known value, while one baseline run found the best known value exactly: the proposed method is more reliable, but not always better in the single best run. (iii) On easy instances (af-1, af-5, af-5-fs, af-7b-fs) the extra LPs make ETiGood up to about 2× worse. (iv) Parameters were set once, but on the same instances that are reported. (v) 3 × 20 runs is fewer than the 500 runs of [1]. (vi) The diet bound proves near-optimality only on af-1.

7. Conclusion and Future Scope

IA-1 showed that random-multistart SLP on the qq-formulation [1], as implemented by our group, is fast but wastes many restarts on poor or infeasible basins on the harder feed-mill instances. In IA-2 we kept the IA-1 model and SLP files unchanged and added three LP sub-problems that follow from the bilinear structure: a diet relaxation, a product LP with bins fixed and a bin LP with bin shares fixed. RG-ILS-SLP uses them to build cost-aware feasible starts, polish every SLP end point and guide restarts by product regret, with short SLP probes to choose among candidate moves.

Over 3 seeds × 20 runs on all 11 instances, mean %Feas rose from 87.9% to 96.1% and mean %Good from 43.6% to 62.0%. On the hard instances %Good rose from 6.4% to 33.9% (paper: 14.8%), and the expected time to a Good solution fell by 5.0×. The gains are statistically significant on 5 instances for %Good and 9 for solution gaps. af-6-f remains unsolved by both methods. Structure-aware restarts are therefore a cheap and effective way to improve SLP for fixed-demand pooling, at the cost of some overhead on easy instances.

7.1 Future Scope

- Use more seeds or 500 runs to match the statistical power of [1], and study why af-6-f resists both methods.
- Choose adaptively between recipe restarts and ILS moves (e.g. a bandit rule), since the ablation shows recipe starts are stronger on af-6-fs and af-7b and the ILS on af-3 and af-4.
- Run the 4 probes in parallel and skip the extra LPs on easy instances to remove the overhead.
- Strengthen the diet bound with McCormick or pq-type cuts to obtain certified gaps on the harder instances.
- Extend to bin capacities, raw-material availability and bin-to-bin flows (generalised pooling).

References

[1] A. Grothey and K. McKinnon, “On the effectiveness of sequential linear programming for the pooling problem,” *Annals of Operations Research*, vol. 322, pp. 691–711, 2023, doi: 10.1007/s10479-022-05156-7.

[2] M. Terzi, Y. Ouazene, A. Yalaoui and F. Yalaoui, “Matheuristics vs. metaheuristics for joint lot-sizing and dynamic pricing problem with nonlinear demands,” *Computers & Operations Research*, vol. 163, 106507, 2024, doi: 10.1016/j.cor.2023.106507.

[3] S. Sass, A. Mitsos, D. Bongartz, I. H. Bell, N. I. Nikolov and A. Tsoukalas, “A branch-and-bound algorithm with growing datasets for large-scale parameter estimation,” *European Journal of Operational Research*, vol. 316, pp. 36–45, 2024, doi: 10.1016/j.ejor.2024.02.020.

[4] P. E. Gill and M. Zhang, “A projected-search interior-point method for nonlinearly constrained optimization,” *Computational Optimization and Applications*, vol. 88, pp. 37–70, 2024, doi: 10.1007/s10589-023-00549-1.

[5] M. Muehlebach and M. I. Jordan, “Accelerated first-order optimization under nonlinear constraints,” *Mathematical Programming*, vol. 215, pp. 407–452, 2026, doi: 10.1007/s10107-025-02224-1.

[6] D. di Serafino, W. W. Hager, G. Toraldo and M. Viola, “On the stationarity for nonlinear optimization problems with polyhedral constraints,” *Mathematical Programming*, vol. 205, pp. 107–134, 2024, doi: 10.1007/s10107-023-01979-9.

[7] R. Misener and C. A. Floudas, “Advances for the pooling problem: modeling, global optimization, and computational studies,” *Applied and Computational Mathematics*, vol. 8, pp. 3–22, 2009.

[8] C. A. Haverly, “Studies of the behavior of recursion for the pooling problem,” *ACM SIGMAP Bulletin*, no. 25, pp. 19–28, 1978.

[9] C. Audet, J. Brimberg, P. Hansen, S. Le Digabel and N. Mladenović, “Pooling problem: alternate formulations and solution methods,” *Management Science*, vol. 50, no. 6, pp. 761–776, 2004.

[10] A. Grothey and K. McKinnon, “Pooling problem test data,” 2020. [Online]. Available: https://www.maths.ed.ac.uk/~agr/PoolingProbs/

[11] R. Fletcher and S. Leyffer, “Nonlinear programming without a penalty function,” *Mathematical Programming*, vol. 91, pp. 239–269, 2002.

[12] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, vol. 10, pp. 119–142, 2018.

[13] H. R. Lourenço, O. C. Martin and T. Stützle, “Iterated local search: framework and applications,” in *Handbook of Metaheuristics*, 3rd ed., Springer, 2019, pp. 129–168.

[14] D. Arthur and S. Vassilvitskii, “k-means++: the advantages of careful seeding,” in *Proc. 18th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.

[15] Group OPT-09 (R. Tambe, S. Tripathi, V. Sharma), “Sequential Linear Programming for Nonlinear Pooling Optimization Problems: a literature review and re-implementation,” IA-1 report, Introduction to Optimization Techniques, K. J. Somaiya College of Engineering, 2026.

Appendix A – Additional Experimental Results

Table A1 gives %Feas / %Good for every seed, with the mean ± standard deviation and the worst objective over feasible runs. Table A2 lists further metrics, including the time to the first Good run (mean over the seeds that found one). Table A3 shows the bin compositions of the best af-3 solution found by the proposed method.

Table A1. %Feas / %Good per seed, and objective over feasible runs

Instance	Method	seed 12345	seed 2024	seed 7	Mean f ± sd	Worst f
af-1	IA-1 (baseline)	100 / 100	100 / 100	100 / 100	296.50 ± 0.12	296.94
af-1	IA-2 (proposed)	100 / 100	100 / 100	100 / 100	296.53 ± 0.12	297.03

Instance	Method	seed 12345	seed 2024	seed 7	Mean f ± sd	Worst f
af-2	IA-1 (baseline)	95 / 0	100 / 0	100 / 5	241,114.95 ± 854.20	245,617.01
af-2	IA-2 (proposed)	95 / 65	95 / 70	100 / 70	239,934.43 ± 1,098.58	246,662.81
af-3	IA-1 (baseline)	100 / 0	95 / 0	95 / 5	123,595.81 ± 296.91	124,339.41
af-3	IA-2 (proposed)	100 / 15	100 / 45	100 / 35	123,255.77 ± 960.33	128,868.28
af-4	IA-1 (baseline)	60 / 25	80 / 20	65 / 15	186,650.18 ± 291.56	187,534.68
af-4	IA-2 (proposed)	80 / 60	90 / 60	90 / 55	186,522.01 ± 202.44	187,553.04
af-5	IA-1 (baseline)	100 / 100	95 / 95	100 / 100	124,066.10 ± 46.96	124,191.73
af-5	IA-2 (proposed)	100 / 100	100 / 100	100 / 95	124,048.25 ± 65.93	124,458.16
af-5-fs	IA-1 (baseline)	90 / 90	70 / 70	80 / 80	119,037.57 ± 31.26	119,152.62
af-5-fs	IA-2 (proposed)	100 / 100	100 / 95	100 / 100	119,050.87 ± 136.92	120,077.56
af-6-f	IA-1 (baseline)	65 / 5	90 / 0	65 / 0	3,102.49 ± 8.89	3,129.79
af-6-f	IA-2 (proposed)	80 / 0	75 / 0	65 / 0	3,107.95 ± 18.72	3,202.37
af-6-fs	IA-1 (baseline)	90 / 0	90 / 0	90 / 10	3,069.67 ± 816.37	8,953.83
af-6-fs	IA-2 (proposed)	100 / 5	100 / 60	100 / 15	2,942.84 ± 55.69	3,334.78
af-7	IA-1 (baseline)	75 / 5	60 / 10	75 / 15	152,505.40 ± 540.13	153,448.22
af-7	IA-2 (proposed)	100 / 20	100 / 10	100 / 25	152,453.00 ± 604.95	153,823.73
af-7b	IA-1 (baseline)	95 / 70	95 / 70	95 / 65	147,711.93 ± 561.33	149,222.87
af-7b	IA-2 (proposed)	100 / 85	100 / 80	100 / 80	147,595.32 ± 527.85	149,768.72
af-7b-fs	IA-1 (baseline)	100 / 100	100 / 95	90 / 90	130,348.93 ± 139.40	131,381.30
af-7b-fs	IA-2 (proposed)	100 / 100	100 / 100	100 / 100	130,321.80 ± 35.97	130,521.79

Table A2. Further metrics (pooled over 3 seeds)

Instance	Method	Ti	%Feas	%Good	%Good range	It	ETiGood	Time to 1st Good (s)	Median gap %
af-1	IA-1 (baseline)	0.41	100.0	100.0	100–100	80.4	0.4	0.5	0.002
af-1	IA-2 (proposed)	0.55	100.0	100.0	100–100	55.1	0.6	0.5	0.002
af-2	IA-1 (baseline)	1.00	98.3	1.7	0–5	110.4	60.1	2.0	0.616
af-2	IA-2 (proposed)	2.15	96.7	68.3	65–70	113.9	3.1	8.3	0.081
af-3	IA-1 (baseline)	1.01	96.7	1.7	0–5	73.4	60.9	2.2	1.053
af-3	IA-2 (proposed)	2.22	100.0	31.7	15–45	81.2	7.0	10.5	0.965
af-4	IA-1 (baseline)	3.37	68.3	20.0	15–25	85.8	16.9	5.7	0.241

Instance	Method	Ti	%Feas	%Good	%Good range	It	ETiGood	Time to 1st Good (s)	Median gap %
af-4	IA-2 (proposed)	6.27	86.7	58.3	55–60	126.6	10.7	8.4	0.199
af-5	IA-1 (baseline)	1.28	98.3	98.3	95–100	118.2	1.3	1.1	0.034
af-5	IA-2 (proposed)	2.73	100.0	98.3	95–100	120.5	2.8	1.6	0.023
af-5-fs	IA-1 (baseline)	3.81	80.0	80.0	70–90	254.6	4.8	4.2	0.009
af-5-fs	IA-2 (proposed)	5.31	100.0	98.3	95–100	230.0	5.4	3.7	0.006
af-6-f	IA-1 (baseline)	6.16	73.3	1.7	0–5	162.2	369.6	96.7	0.927
af-6-f	IA-2 (proposed)	10.15	73.3	0.0	0–0	194.8	∞	∞	0.952
af-6-fs	IA-1 (baseline)	12.28	90.0	3.3	0–10	196.0	368.3	65.9	0.906
af-6-fs	IA-2 (proposed)	13.58	100.0	26.7	5–60	155.4	50.9	151.3	0.284
af-7	IA-1 (baseline)	7.26	70.0	10.0	5–15	206.8	72.6	52.1	0.788
af-7	IA-2 (proposed)	7.95	100.0	18.3	10–25	119.7	43.4	49.0	0.778
af-7b	IA-1 (baseline)	10.70	95.0	68.3	65–70	143.1	15.7	20.5	0.148
af-7b	IA-2 (proposed)	9.95	100.0	81.7	80–85	109.5	12.2	10.3	0.111
af-7b-fs	IA-1 (baseline)	58.52	96.7	95.0	90–100	120.2	61.6	48.7	0.030
af-7b-fs	IA-2 (proposed)	65.30	100.0	100.0	100–100	104.1	65.3	99.0	0.020

Table A3. Bin compositions (% of each raw material) of the best af-3 solution of IA-2 (f = 122267.31)

Raw material	MI01	MI02	MI03	MI04	MI05	MI06	MI07
RM04	10.0	10.0	10.0	10.0	9.5		8.4
RM05			27.7				
RM06						15.5	18.2
RM10	2.4	62.1				36.8	
RM12	78.2	22.5	43.4	29.2	76.0		64.0
RM14			3.1	51.1		39.0	
RM15	4.0	4.0	4.0	4.0	4.0		4.0
RM18	4.0	0.4	1.7	4.7	3.3	2.7	1.6
RM19						1.3	
RM20	1.0	1.0	1.0	1.0	1.0	1.0	1.0
RM21					3.0		
RM22	0.4				1.4	0.2	0.3
RM23					1.7	2.5	2.5
RM31			9.0			1.0	
tonnage served	241	353	46	197	157	35	479

Appendix B – Source Code / Supporting Material

All final IA-2 deliverables are in the group's Google Drive folder (access: anyone with the link – viewer): [paste the Google Drive folder link]. Source code online (GitHub, with a runnable Google Colab notebook IA2_Demo_Colab.ipynb): [paste the GitHub repository link]. The folder structure is:

```
OPT-09_IA2/
├── 01_Research_Paper/    Research_Paper_Draft.pdf (+ .docx)
├── 02_PPT/               Presentation.pdf, Presentation.pptx
├── 03_Poster/            Poster.pdf
├── 04_Source_Code/       qq_model.py, slp.py (IA-1, unchanged), run_experiments.py (extended),
│                         rg_ils.py (new), analyze_results.py, make_flowchart.py, run_all.sh,
README.md
├── 05_Dataset/           af-1.txt ... af-7b-fs.txt
├── 06_Results/           Result_Tables.xlsx, Graphs/, baseline|proposed|abl_*/seed*/runs_*.csv, logs/
└── 07_Supporting_Material/  IA-1 report, form answers, flowchart
```